Sim-to-Real Guide: Plate Sorting on NEURA MAiRA

Running the dirty/clean plate sorter on the real robot, without RoboDK

Read this first. Everything was built and tested only in simulation. The real robot has never run these programs. Test at low speed with the emergency stop in reach.

Two things will not work until you do them: (1) the robot programs contain **no gripper commands** (Section 4), and (2) they contain **no speed setting** (Section 5).

1. Files in the package

File	What it is
main_real.py	The main program. Takes a camera image, asks the neural network 'clean or dirty', then runs D1() or C1().
D1_program.py	Robot program (NeuraPy, exported from RoboDK). Moves the plate to the **dishwasher rack** (dirty).
C1_program.py	Robot program (NeuraPy, exported from RoboDK). Moves the plate to the **clean rack**.
plate_cleanliness.weights.h5	Trained neural network weights. Must be in the same folder as main_real.py.
requirements.txt	Python libraries and versions used for the network (TensorFlow, OpenCV, numpy).

All files go in **one folder**. The RoboDK simulation script (Dirty vs clean plate 1) is **not** used on the real robot.

2. What you need

Item	Notes
NEURA MAiRA robot, powered, emergency stop tested	Workspace clear. Tool/gripper mounted.
Windows PC connected to the robot	**RoboDK is not needed.**
NeuraPy (NEURA's Python SDK)	Downloaded from the NEURA Partner Hub. NEURA's manual says this needs credentials from your NEURA business partner or project manager.
Python	NEURA's manual describes Python 3.8 (3.6 or newer should work). **Open issue:** the pinned libraries in requirements.txt (numpy 2.2 and newer TensorFlow) need **Python 3.10 or newer**. Ask NEURA whether NeuraPy works with that Python version.
A camera the PC can read	USB or network. Set CAMERA_SOURCE in main_real.py.
Same physical setup as the simulation	Table, plate pick-up spot, both racks, camera position.

3. Step-by-step

Step 1: Install
Install Python and NeuraPy as described by NEURA (Section 2). Then, in a terminal in the package folder: `pip install -r requirements.txt`. Confirm that NeuraPy can connect to the robot with NEURA's own example before continuing.

Step 2: Camera
Open main_real.py and set CAMERA_SOURCE (0 = first USB camera). The script saves the last image as last_capture.png so you can see what the network saw. The image should look like the images the

network was trained on (angle, light, distance).

Step 3: Test each robot program alone
Reduce the speed first (Section 5). Put a plate in place. Run `python D1_program.py`, watch the whole motion, then `python C1_program.py`. Check the start pose, the pick-up, and the end at the correct rack. All targets are fixed joint angles, so real positions must match the simulation. If the arm misses, stop and tell the sender.

Step 4: Full run
Run `python main_real.py`. It shows its decision and waits for Enter before the robot moves (CONFIRM_BEFORE_MOVE). Test one clean and one dirty plate.

Step 5: Faster
Raise speed in small steps, only after several correct runs.

4. Gripper: not included in the robot programs

Both robot programs contain only movement commands. The lines `# Attach to Tool 1` and `# Detach from Tool 1` are **comments** that only mean something inside RoboDK. On the real robot, **nothing grips or releases the plate**. The gripper (open/close or vacuum on/off) must be added at those two places in D1_program.py and C1_program.py, using NEURA's method for the gripper you use. The sender does not know your gripper, so this must be done on site.

5. Speed

The exported programs set **no speed**, so the robot uses its default. Set a reduced speed before the first run. NEURA's software manual describes speed scaling in its programming interface. I could not confirm which speed option exists in NeuraPy's move commands, so check the NeuraPy documentation from the Partner Hub or ask NEURA support. Do not run at full speed on the first tests.

6. Safety

• Emergency stop within reach for every run.
• Nobody inside the robot's workspace while it moves.
• The programs use joint moves only. The path between points is not checked for collisions with the table or racks. Watch the first runs closely.
• If the arm moves somewhere unexpected, press the emergency stop.

7. Troubleshooting

Problem	Check
main_real.py cannot import D1 or C1	File names must be exactly D1_program.py and C1_program.py, in the same folder.
Error about Robot() or connection	NeuraPy not installed or robot not reachable. Each exported file creates its own Robot(); ask NEURA if two are allowed, otherwise share one.
Weights file error	TensorFlow/Keras version differs from the one used in training. Match the versions in requirements.txt.
Camera error	Wrong CAMERA_SOURCE or camera in use by another program.
Wrong clean/dirty decisions	Real camera image differs from the training images. If the network was trained on simulated images, it may need retraining with real photos.
Arm misses plate or rack	Real positions differ from the simulation.

Sender checklist and details

Complete before sending. Items marked (?) are things I could not know.

Item	Answer
Robot	Neura MAiRA M (post processor: Neura)
Exported files renamed to D1_program.py / C1_program.py?	
requirements.txt created (pip freeze; keep tensorflow, opencv-python, numpy)?	
Camera used for training: simulated (RoboDK) or real photos? (?)	
Home / start pose: does C1 also start and end at the same pose as D1? (?)	
Gripper type and how it is controlled (?)	
Python version used for the network	
Robot IP / how NeuraPy connects (from NEURA documentation) (?)	
Contact (name, email, time zone)	